

Project 2: Automated Adversary Emulation & SIEM Detections

Module 1: Environment Setup & API Connection

Workspace & API Setup

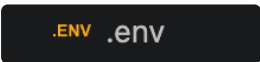
To get started on Project 2, I created a Python project directory on my Kali Linux attack machine (10.1.4.100) and connected to it from VS Code on my laptop using the Remote-SSH extension. This gave me a full code editor on my desktop while executing everything directly inside Kali.



I set up a Python virtual environment to keep packages isolated and generated a dedicated REST API token inside Splunk Web.

Name	Status	Username	Audience	Issued At	Expiration	Last Used	
	Enabled	admin	SOC Automation Script	8/10/2026 3:09:19 PM CEST	9/9/2026 3:09:19 PM CEST	Unused	

To keep credentials safe, I stored the token and host IP addresses inside a .env file rather than hardcoding them into my source code.



Troubleshooting Network Isolation (Port 9997 vs 8089)

When I ran my initial test script (test_splunk_conn.py) to query Splunk at 10.1.1.10:8089 , the request timed out.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python test_splunk_conn.py
[ERROR] Connection failed: HTTPConnectionPool(host='10.1.1.10', port=8089): Max retries exceeded with url: /service/s/authentication/users?output_mode=json (Caused by ConnectTimeoutError(<HTTPSConnection(host='10.1.1.10', port=8089) at 0x7fc624492e40>, 'Connection to 10.1.1.10 timed out. (connect timeout=5)'))
```

I realized this was due to the pfSense boundary rules I created in Project 1. While log telemetry streams into Splunk over port 9997, Splunk uses port 8089 for REST API management and searches. Because the Attack Zone is isolated from internal networks, pfSense dropped the API traffic before it could reach the SIEM.

Fixing the Firewall Rule & Verifying Success

I created a new pfSense rule intended to allow traffic from my Kali machine's subnet over port 8089.

States	Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description
0/0 B	IPv4 TCP	OPT3 address	*	10.1.1.10	8089	*	none		Allow Kali to query Splunk REST API

However, when I tested the connection with netcat, it was still timing out.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ nc -zvw3 10.1.1.10 8089
10.1.1.10: inverse host lookup failed: Unknown host
(UNKNOWN) [10.1.1.10] 8089 (?): Connection timed out
```

After some troubleshooting, I found out that the new pass rule I created for port 8089 had its source set to `OPT3` address (which only matches the pfSense gateway interface IP) instead of `OPT3 subnets` (which covers my Kali machine). Because Kali's IP didn't match, the traffic fell through to my default rule blocking all private subnets.

I edited the rule source to `OPT3 subnets`, saved, and applied the changes.

Protocol	Source	Port	Destination	Port	Gateway	Queue	Schedule	Description
IPv4 TCP	OPT3 subnets	*	10.1.1.10	8089	*	none		Allow Kali to query Splunk REST API

Re-testing with netcat immediately showed an open port, and running `python test_splunk_conn.py` returned a clean `[SUCCESS]` response.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ nc -zv 10.1.1.10 8089
10.1.1.10: inverse host lookup failed: Unknown host
(UNKNOWN) [10.1.1.10] 8089 (?) open

(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python test_splunk_conn.py
[SUCCESS] Connected to Splunk REST API successfully!
```

Module 2: Automated Attack Simulation & SIEM Detection Audit

Building the Attack Script (`attack_module.py`)

Next, I created `attack_module.py` to simulate two different attack techniques from my Kali machine (`10.1.4.100`):

- Web Path Traversal:** Sends an HTTP GET request to DVWA on `vuln-web` (`10.1.2.10`) targeting `/vulnerabilities/fi/?page=../../../../etc/passwd`. This simulates a Local File Inclusion (LFI) attack trying to read local system files.
- Covert DNS Exfiltration:** Encodes a sensitive text string into Base32 and embeds it inside a DNS subdomain query (like `INHU4RSJIRCU4VCJIFGF.exfil.lab`). To ensure the request always goes directly to pfSense (`10.1.4.1`) regardless of Kali's system DNS configuration, I had the script send a raw UDP DNS packet directly to port 53.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python attack_module.py
=== Step 1: Executing Attack Simulation Module ===
[*] Triggering Web Attack (Path Traversal)...
Target: http://10.1.2.10/vulnerabilities/fi/?page=../../../../etc/passwd
[+] HTTP Status Code: 200

[*] Triggering Covert Channel (DNS Exfiltration)...
Exfiltrating domain query: INHU4RSJIRCU4VCJIFGF.exfil.lab
[+] Query sent to gateway (DNS resolution failed as expected).

=== Attack Simulation Completed ===
```

Automating SIEM Verification (`splunk_module.py`)

Instead of opening Splunk Web and manually checking for logs after every test, I wrote `splunk_module.py` to audit detections automatically over the REST API.

The script uses the API token in my `.env` file to connect to `[https://10.1.1.10:8089]` (`https://10.1.1.10:8089`) and runs fast `oneshot` searches for events from the last 5 minutes:

- `index=* "etc/passwd"` to find the web path traversal logs.
- `index=* "exfil.lab"` to find pfSense DNS query logs.

When Splunk finds a match, the script prints `[SUCCESS]` , counts the matching events, and extracts the attacker's source IP (`10.1.4.100`) so we can pass it down the pipeline.

Troubleshooting Missing DNS Logs in pfSense

When I tested the pipeline, the web path traversal attack worked right away—it returned an HTTP `200` response and Splunk caught the log. However, the DNS exfiltration check kept returning `[FAILURE]` .

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python splunk_module.py
=== Step 2: Executing Splunk SIEM Detection Verification ===

[*] Auditing Splunk SIEM for: Web Path Traversal Attack...
[SUCCESS] Splunk caught the attack! Found 1 matching log event(s).
[+] Extracted Source/Attacker IP: 10.1.4.100

[*] Auditing Splunk SIEM for: Covert DNS Exfiltration...
[FAILURE] Detection failed! No matching events found in Splunk.

=== Splunk Detection Audit Completed ===
```

Checking pfSense's log viewer revealed that the DNS resolver hadn't recorded any new logs in weeks. To get DNS query telemetry flowing from pfSense into Splunk, I adjusted three settings:

1. **Enabled Query Logging:** Added `log-queries: yes` under **Services > DNS Resolver > General Settings (Custom Options)** and set the log level to `Level 2` under Advanced Settings.
2. **Forwarded DNS Syslog:** Opened **Status > System Logs > Settings** and checked the box for **DNS Events** under Remote Logging Options so pfSense forwarded DNS logs to Splunk over UDP port 1514.
3. **Checked Listening Interfaces:** Confirmed the DNS Resolver was set to listen on all interfaces, including `OPT3` (the Attack Zone subnet).

Custom options

```
server:
log-queries: yes
```

Log Level

Level 2: Detailed operational information

Select the level of detail to be logged. Each level also includes the information from previous levels. The default is basic operational information (level 1)

After restarting the DNS Resolver service and re-running the scripts, Splunk caught both attacks immediately, returning matching log counts and successfully extracting my Kali IP.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python splunk_module.py
=== Step 2: Executing Splunk SIEM Detection Verification ===

[*] Auditing Splunk SIEM for: Web Path Traversal Attack...
[SUCCESS] Splunk caught the attack! Found 1 matching log event(s).
[+] Extracted Source/Attacker IP: 10.1.4.100

[*] Auditing Splunk SIEM for: Covert DNS Exfiltration...
[SUCCESS] Splunk caught the attack! Found 8 matching log event(s).
[+] Extracted Source/Attacker IP: 10.1.4.100

=== Splunk Detection Audit Completed ===
```

Threat Intelligence & Master Pipeline (`pipeline.py`)

To finish the pipeline, I wrote `threat_intel.py` to enrich the extracted IP address, then created `pipeline.py` to run all three steps automatically in a single command.

When `splunk_module.py` finds an attack and pulls out the source IP, it hands it off to `threat_intel.py` :

- **Internal IPs:** If the IP is a private address (like my Kali machine at `10.1.4.100`), the script marks it as an internal lab host instead of making unnecessary external API calls.
- **Public IPs:** If it's an external address, the script queries AbuseIPDB to pull reputation scores, country data, and usage types.

Running `python pipeline.py` executes the whole flow end-to-end: it fires the web and DNS attacks, pauses briefly for Splunk to ingest the logs, checks the SIEM over the API, and enriches the attacker IP in less than 15 seconds.

```
(venv)-(attacker@kali-attacker01)-[~/project2-soc-automation]
$ python pipeline.py

=====
AUTOMATED DETECTION & ENRICHMENT PIPELINE
=====

--- STEP 1: OFFENSIVE ATTACK EMULATION ---
[*] Triggering Web Attack (Path Traversal)...
    Target: http://10.1.2.10/vulnerabilities/fi/?page=../../../../etc/passwd
    [+] HTTP Status Code: 200

[*] Triggering Covert Channel (DNS Exfiltration)...
    Exfiltrating domain query: INHU4RSJIRCU4VCJIFGF.exfil.lab
    [+] Query sent directly to pfSense (10.1.4.1:53) over UDP.

[*] Waiting 8 seconds for Splunk log ingestion...

--- STEP 2: SIEM DETECTION AUDIT ---

[*] Auditing Splunk SIEM for: Web Path Traversal...
    [SUCCESS] Splunk caught the attack! Found 2 matching log event(s).
    [+] Extracted Source/Attacker IP: 10.1.4.100

--- STEP 3: THREAT INTEL ENRICHMENT ---

[*] Enriching Threat Intelligence for IP: 10.1.4.100...
    [INFO] IP 10.1.4.100 is an internal lab address (RFC 1918).
    [+] Threat Assessment: INTERNAL_ATTACK_SOURCE (Kali Attack Machine)

=====
PIPELINE EXECUTION COMPLETE
=====
```

Module 3: Writing & Tuning Splunk Detections (SPL Rules)

Building the LFI & Path Traversal Rule

To finish up Project 2, I wrote custom Splunk searches (SPL rules) to automatically catch web attacks and covert DNS exfiltration.

When I ran my first search for path traversal (`index=linux uri="*etc/passwd*`), Splunk showed 0 events, even though my Python attack script got an HTTP 200 success response from DVWA.

Troubleshooting Container Logs I ran a broader search (`index=linux "passwd"`) to look at the raw log data:

index=linux passwd Time range: Last 15 minutes

✓ 1 event (8/18/26 7:48:44.000 PM to 8/18/26 8:03:44.000 PM) No Event Sampling

Events (1) Patterns Statistics Visualization

Timeline format Zoom Out Zoom to Selection Deselect 1 minute per column

Format Show: 20 Per Page View: List

< Hide Fields All Fields

SELECTED FIELDS

- a host 1
- a source 1
- a sourcetype 1

INTERESTING FIELDS

- # date_hour 1
- # date_mday 1
- # date_minute 1
- a date_month 1
- # date_second 1
- a date_wday 1
- # date_year 1
- # date_zone 1
- a index 1
- # linecount 1
- a log 1
- a punct 1
- a splunk_server 1
- a stream 1
- a time 1
- # timeendpos 1
- # timestartpos 1

+ Extract New Fields

Time Event

8/18/26 8:01:59.000 PM

```
[{"log": "10.1.4.100 - - [18/Aug/2026:18:01:59 +0000] \"GET /vulnerabilities/fi/?page=../../../../etc/passwd HTTP/1.1\" 302 485 \"-\" \"python-requests/2.34.2\""}]
```

stream: stdout
time: 2026-08-18T18:01:59.988667364Z

Show as raw text

Event Actions

Type	Field	Value	Actions
Selected	host	vuln-web	
	source	/var/lib/docker/containers/6985d1765241b26ecf31493059568e3a5b9b4d62f4d7633a0ea0515b2b1f49d0/6985d1765241b26ecf31493059568e3a5b9b4d62f4d7633a0ea0515b2b1f49d0.json.log	
	sourcetype	docker:json	
Event	log	10.1.4.100 - - [18/Aug/2026:18:01:59 +0000] \"GET /vulnerabilities/fi/?page=../../../../etc/passwd HTTP/1.1\" 302 485 \"-\" \"python-requests/2.34.2\"	
	stream	stdout	
	time	2026-08-18T18:01:59.988667364Z	
Time	_time	2026-08-18T20:01:59.000+02:00	
Default	index	linux	
	linecount	1	

- I noticed DVWA runs inside a Docker container, so Splunk ingests its logs under `sourcetype="docker:json"`.
- The web request was nested inside a JSON field named `log`. Because Splunk hadn't automatically parsed fields like `uri` or `src_ip`, standard field searches didn't work.

To fix this, I added inline regular expressions (`rex`) to pull the attacker IP, HTTP method, and requested URI straight out of the raw container log on the fly.

New Search SPL Convert to SPL2 Save As Create Table View Close

Time range: Last 15 minutes

```
index=linux sourcetype="docker:json" ("*etc/passwd*" OR "*../*" OR "%2e%2e%2f*")
| rex field=log "(?<src_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}).*?(?<http_method>\w+)\s+(?<uri>\S+)"
| stats count earliest(_time) as first_seen latest(_time) as last_seen by src_ip, host, http_method, uri
| fieldformat first_seen=strftime(first_seen, "%Y-%m-%d %H:%M:%S")
| fieldformat last_seen=strftime(last_seen, "%Y-%m-%d %H:%M:%S")
```

✓ 1 event (8/18/26 7:50:00.000 PM to 8/18/26 8:05:00.000 PM) No Event Sampling

Events Patterns Statistics Visualization

Show: 20 Per Page Format Preview: On

src_ip	host	http_method	uri	count	first_seen	last_seen
10.1.4.100	vuln-web	GET	/vulnerabilities/fi/?page=../../../../etc/passwd	1	2026-08-18 20:01:59	2026-08-18 20:01:59

```
index=linux sourcetype="docker:json" ("*etc/passwd*" OR "*../*" OR "%2e%2e%2f*")
| rex field=log "(?<src_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}).*?(?<http_method>\w+)\s+(?<uri>\S+)"
| stats count earliest(_time) as first_seen latest(_time) as last_seen by src_ip, host, http_method, uri
```

```
| fieldformat first_seen=strftime(first_seen, "%Y-%m-%d %H:%M:%S")
| fieldformat last_seen=strftime(last_seen, "%Y-%m-%d %H:%M:%S")
```

MITRE ATT&CK Mapping: T1190 (Exploit Public-Facing Application) & T1083 (File and Directory Discovery)

Building the DNS Exfiltration Rule

Next, I created a search to flag encoded DNS queries targeting my lab domain (`.exfil.lab`).

My initial search (`index=pfsense sourcetype="pfsense:unbound"`) also returned **0 events**.

The screenshot shows the Splunk Search interface. The search bar contains the query: `index=pfsense sourcetype="pfsense:unbound" "exfil.lab"`. Below the search bar, it indicates **0 events** found for the time range from 8/18/26 7:50:53.000 PM to 8/18/26 8:05:53.000 PM. The interface includes tabs for Events, Patterns, Statistics (0), and Visualization. The Statistics tab is active, showing a message: "No results found. Try expanding the time range."

Troubleshooting pfSense Telemetry

I searched across all indexes (`index=* "exfil.lab"`) to see how old logs were formatted:

The screenshot shows the Splunk Search interface with the query `index=* "exfil.lab"`. It indicates **16 events** found. The Events tab is active, showing a list of events. The first event is from 8/14/26 9:40:13.000 AM, with the message: "Aug 14 07:40:13 10.1.1.1 Aug 14 09:40:13 unbound[92565]: [92565:0] info: 10.1.4.100 INHU4RSJIRC4VCJIFGF.exfil.lab. A IN". The event details show: host = 10.1.1, source = udp:1514, and sourcetype = syslog.

i	Time	Event
>	8/14/26 9:40:13.000 AM	Aug 14 07:40:13 10.1.1.1 Aug 14 09:40:13 unbound[92565]: [92565:0] info: 10.1.4.100 INHU4RSJIRC4VCJIFGF.exfil.lab. A IN host = 10.1.1 source = udp:1514 sourcetype = syslog
>	8/14/26 9:40:13.000 AM	Aug 14 07:40:13 10.1.1.1 Aug 14 09:40:13 unbound[92565]: [92565:0] info: 10.1.4.100 INHU4RSJIRC4VCJIFGF.exfil.lab. A IN host = 10.1.1 source = udp:1514 sourcetype = syslog

- I realized pfSense sends Unbound DNS logs to Splunk under `sourcetype=syslog` (not `pfsense:unbound`).
- Telemetry had stopped arriving because the pfSense `syslogd` and `unbound` services had stalled.

I logged into the pfSense WebGUI, went to **Status > Services**, and restarted both `syslogd` and `unbound`. After re-running my attack script from Kali, new logs immediately showed up in Splunk.

New Search SPL Convert to SPL2 Save As Create Table View Close

```
index=* sourcetype=syslog "exfil.lab"
| rex field=_raw "info:\s+(?<client_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\s+(?<query_domain>[A-Za-z0-9\=\.]+\.\exfil\.lab)"
| stats count, values(client_ip) as attacker_ip by query_domain
```

Time range: All time Q +

✓ 16 events (before 8/18/26 8:21:50.000 PM)
No Event Sampling
Job
II

↶
↷
⬇
💡 Smart Mode

Events
Patterns
Statistics (1)
Visualization

Show: 20 Per Page
Format
Preview: On

query_domain	count	attacker_ip
INHU4RSJIRC4VCJIFGF.exfil.lab	6	10.1.4.100

```
index=* sourcetype=syslog "exfil.lab"
| rex field=_raw "info:\s+(?<client_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\s+(?<query_domain>[A-Za-z0-9\=\.]+\.\exfil\.lab)"
| stats count, values(client_ip) as attacker_ip by query_domain
```

MITRE ATT&CK Mapping: T1071.004 (Application Layer Protocol: DNS) & T1048.003 (Exfiltration Over Alternative Protocol)

Project 2 Wrap-Up & Key Takeaways

To keep my detection rules saved and version-controlled, I created a `queries/` folder in my GitHub repository and added both files:

- `queries/lfi_detection.spl`
- `queries/dns_exfiltration.spl`

With these rules tested and working, Project 2 is officially complete! The entire pipeline now handles automated attack simulation, SIEM detection checks via the Splunk REST API, threat intelligence lookup via AbuseIPDB, and production-ready SPL correlation rules.

Visualizing Telemetry & Detections in Splunk Dashboard Studio

To pull my infrastructure telemetry and real-time threat detections into a single pane of glass, I built a custom dashboard in Splunk Dashboard Studio (`SOC Telemetry & Threat Detection`).

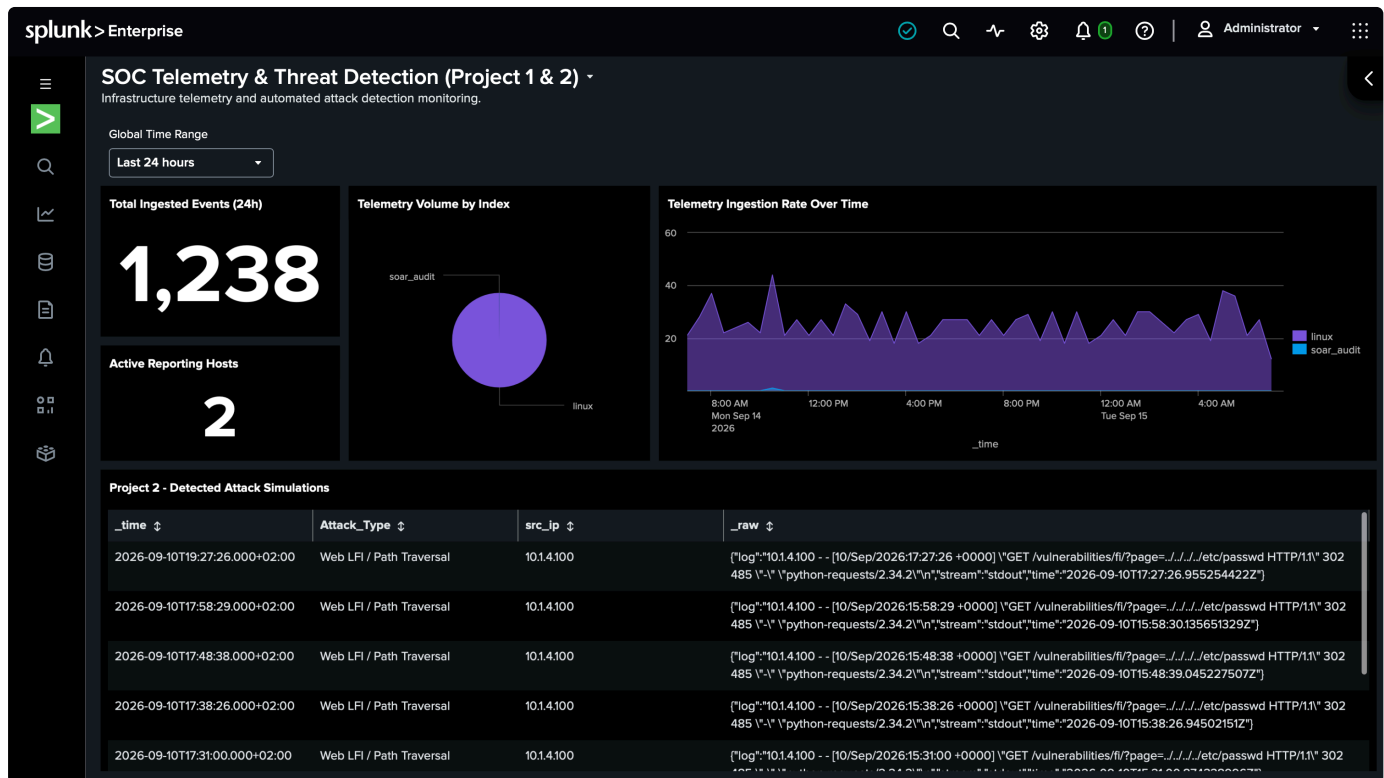
I used the **Absolute** layout mode so I could position KPI tiles, charts, and detection tables cleanly without strict grid constraints.

What I Configured on the Dashboard:

- Total Ingested Events (24h) & Active Reporting Hosts:** Single-value KPI panels tracking total volume (`index=* | stats count`) and active endpoint counts (`index=* | stats dc(host)`).
- Telemetry Volume & Ingestion Rate:** A pie chart and timechart (`index=* | timechart count by index`) tracking log volume across my `linux` and `soar_audit` indexes.
- Project 2 Attack Detections Table:** A live detection table catching automated LFI and DNS exfiltration attacks. I used inline regex to extract the attacker source IP (`10.1.4.100`) directly out of raw Docker JSON container logs on the fly:

```
(index=* "etc/passwd") OR (index=* "*.exfil.lab")
| rex field=_raw "\"log\": \"(?<src_ip>\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})"
| eval Attack_Type=if(like(_raw, "%etc/passwd%"), "Web LFI / Path Traversal", "Covert DNS
```

```
Exfiltration")  
| table _time, Attack_Type, src_ip, _raw
```



Key Takeaways & Lessons Learned

- **Log Ingestion & Parsing:** Learned that containerized applications (like Dockerized DVWA) nest logs inside JSON wrapper fields (`sourcetype="docker:json"`), requiring inline regex (`rex`) to extract field values like `src_ip` and `uri`.
- **Network & Gateway Troubleshooting:** Identified how pfSense handles DNS telemetry under `sourcetype=syslog` rather than standard unbound sourcetypes, and solved ingestion freezes by restarting gateway services (`syslogd` / `unbound`).
- **Closed-Loop SOC Automation:** Built a complete, end-to-end Python pipeline (`pipeline.py`) that executes an attack, audits the SIEM via API, and enriches attacker IPs in under 15 seconds.

View Full Source Code & Repository:

The complete Python scripts, configuration templates, and SPL queries are available on my GitHub:

<https://github.com/antonvikstrom/automated-threat-detection-pipeline>